



Complete Guide: OpenTelemetry Capabilities & Playwright Integration Architecture

Document Overview: This comprehensive guide details the types of backend issues OpenTelemetry (OTel) detects and reports, and provides an end-to-end technical architectural explanation of integrating Playwright frontend testing with OpenTelemetry backend observability.



Table of Contents

1. [Executive Summary](#executive-summary) 2. [Part 1: Issues OpenTelemetry (OTel) Handles & Reports] (#part-1-issues-opentelemetry-otel-handles--reports) - [1. Slow Database Queries & SQL Performance](#1-slow-database-queries--sql-performance) - [2. Database Connection Pool Exhaustion](#2-database-connection-pool-exhaustion) - [3. Unhandled Server Exceptions & Code Line Stack Traces](#3-unhandled-server-exceptions--code-line-stack-traces) - [4. Third-Party API & External Service Failures](#4-third-party-api--external-service-failures) - [5. Microservice Dependency & Chain Failures](#5-microservice-dependency--chain-failures) - [6. Authentication & Security Middleware Rejections](#6-authentication--security-middleware-rejections) 3. [Part 2: Full-Stack Integration Architecture (Playwright + OpenTelemetry)] (#part-2-full-stack-integration-architecture-playwright--opentelemetry) - [Step 1: Playwright Frontend Execution](#step-1-playwright-frontend-execution) - [Step 2: W3C `traceparent` Header Injection](#step-2-w3c-traceparent-header-injection) - [Step 3: Backend Trace Propagation & Span Recording](#step-3-backend-trace-propagation--span-recording) - [Step 4: OTLP Ingestion into Flask & Supabase](#step-4-otlp-ingestion-into-flask--supabase) - [Step 5: Full-Stack Correlation & Gantt Chart Waterfall](#step-5-full-stack-correlation--gantt-chart-waterfall) 4. [Part 3: Grounded AI Error Explanation Engine](#part-3-grounded-ai-error-explanation-engine) 5. [Part 4: Summary Table & Capabilities Matrix](#part-4-summary-table--capabilities-matrix)



Executive Summary

Traditional QA automation tests applications from the **outside only** (the browser interface). When a test step fails with a generic error like `HTTP 500 Internal Server Error`, frontend tools cannot explain why the server crashed.

By integrating **Playwright (Frontend Automation)** with **OpenTelemetry (Backend Observability)**: - **Playwright** automates browser actions, captures screenshots, monitors browser console logs, and records network HTTP status codes. - **OpenTelemetry** monitors internal backend code, database query execution times, microservice hops, and server memory. - **W3C `traceparent` Header Correlation** links every Playwright browser click directly to its corresponding backend database query and code line.

Part 1: Issues OpenTelemetry (OTel) Handles & Reports

OpenTelemetry monitors server code execution in real-time and reports exact root causes back to developers and QA engineers.

1. Slow Database Queries & SQL Performance

- **The Problem:** An API request takes 5+ seconds to complete. - **What OTel Reports:** - Pinpoints the exact SQL query causing the delay (e.g. ``SELECT * FROM orders WHERE customer_id = X``). - Reports query execution time (e.g. ``4,850ms``). - Explains the cause: Missing database index on ``customer_id``.

2. Database Connection Pool Exhaustion

- **The Problem:** Under heavy test traffic, the database stops responding and drops requests. - **What OTel Reports:** - Reports database connection pool limit reached (``FATAL: connection limit exceeded (max 100 connections)``). - Shows which code file opened the unclosed connection (``db_pool.py:L34``).

3. Unhandled Server Exceptions & Code Line Stack Traces

- **The Problem:** Backend code crashes internally, returning a blank ``500 Server Error`` page. - **What OTel Reports:** - Exact programming language stack trace, file name, method name, and line number. - **Python Example:** ``AttributeError: 'NoneType' object has no attribute 'email' in `users.py:L42``. - **Node.js Example:** ``TypeError: Cannot read property 'id' of undefined` in `authController.js:L88``. - **Java Example:** ``java.lang.NullPointerException` in `PaymentService.java:L105``.

4. Third-Party API & External Service Failures

- **The Problem:** The app fails because an external vendor (Stripe, SendGrid, Twilio, OpenAI, AWS S3) failed or timed out. - **What OTel Reports:** - Reports outbound HTTP status code (e.g., ``429 Too Many Requests`` or ``504 Gateway Timeout``). - Displays third-party API endpoint URL (e.g. ``https://api.stripe.com/v1/charges``).

5. Microservice Dependency & Chain Failures

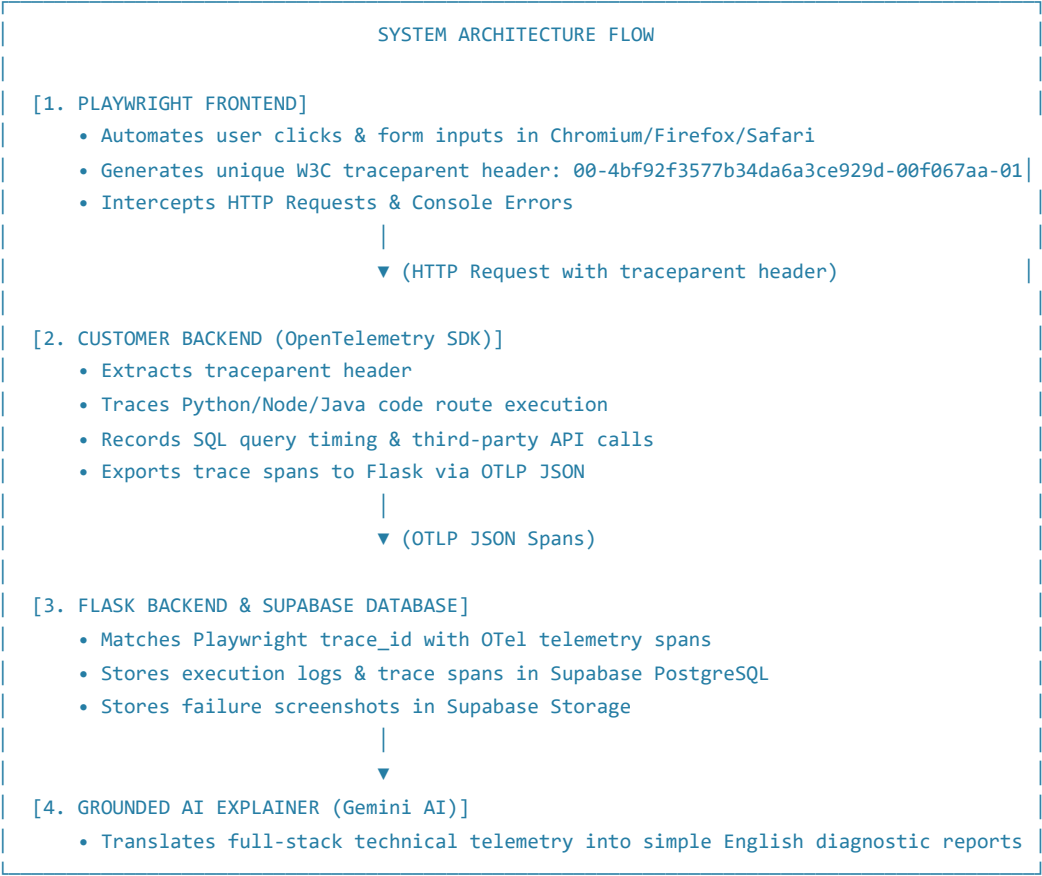
- **The Problem:** In a microservices system, a request passes through multiple services before failing. - **What OTel Reports:** - **Service Map Hops:** ``API Gateway` (10ms)  $\rightarrow$  `Auth Service` (15ms)  $\rightarrow$  `Order Service` (5,000ms TIMEOUT ✗)`. - Highlights the exact microservice where the breakage occurred.

6. Authentication & Security Middleware Rejections

- **The Problem:** Server middleware rejects a user request due to expired security tokens. - **What OTel Reports:** - ``auth_middleware.py:L28``: JWT token signature verification failed (token expired 120 seconds ago).

Part 2: Full-Stack Integration Architecture (Playwright + OpenTelemetry)

Here is how our application integrates Playwright and OpenTelemetry step-by-step:



Detailed Integration Sequence:

1. Step 1: Playwright Frontend Execution

- Playwright executes test steps (opening URLs, filling inputs, clicking buttons).
- Listens to browser events (

page.on('response'), `page.on('console')`).

2. Step 2: W3C `traceparent` Header Injection - For every outbound HTTP request, Playwright injects a W3C Standard Trace Header: `traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01`

3. Step 3: Backend Trace Propagation & Span Recording

- The customer's backend OpenTelemetry SDK reads the

traceparent` header. - All backend operations (SQL queries, microservice hops, code functions) are tagged with this Trace ID.

4. **Step 4: OTLP Ingestion into Flask & Supabase** - Customer OTel SDK exports trace spans to our Flask endpoint: `POST /api/otlp/v1/traces`

- Flask indexes and saves these spans into Supabase PostgreSQL (telemetry_spans`).

5. **Step 5: Full-Stack Correlation & Gantt Chart Waterfall** - The React frontend displays a single unified Gantt chart timeline showing: `User Click (10ms) → API Flight Time (50ms) → Python Route (20ms) → SQL Query (1,200ms) → React Render (30ms)`.

Part 3: Grounded AI Error Explanation Engine

Our platform uses Google Gemini AI to convert complex full-stack telemetry into plain English error reports.

Grounding Rules & Format:

```
```text
=====
FAILED STEP #4: Click Submit
=====

🔴 FRONTEND FINDING (Confirmed by Playwright): The user clicked "Sign In →". The browser sent POST /api/v1/login which failed with HTTP 500 Internal Server Error after 3,240ms.

⚙️ BACKEND FINDING (Confirmed by OpenTelemetry): Request reached AuthService (auth_controller.py:L45). Database query SELECT * FROM users failed with psycopg2.OperationalError: FATAL: connection limit exceeded (max 100 connections).

💡 RECOMMENDED FIX: Increase database connection pool size in PostgreSQL or implement connection pooling (PgBouncer).
=====
```
```

Part 4: Summary Table & Capabilities Matrix

| | | | |
|-----------------------------------|------------------------------------|--------------------------------------|--|
| Issue Type | Detected by Playwright (Frontend)? | Detected by OpenTelemetry (Backend)? | Integrated Platform Benefit |
| | --- --- --- --- | | |
| Broken Button / UI Layout | ✅ YES | ❌ NO | Fails UI step & captures screenshot |
| Wrong Text on Page | ✅ YES | ❌ NO | Verifies page DOM content |
| HTTP 500 Server Error | ✅ YES (sees HTTP 500) | ✅ YES (sees stack trace) | Links browser error directly to code line |
| Slow 4.5s SQL Query | ❌ NO (only sees latency) | ✅ YES | Pinpoints slow SQL query & missing index |
| Database Pool Exhaustion | ❌ NO (only sees generic 500) | ✅ YES | Explains connection pool exhaustion |
| Third-Party API Rate Limit | ❌ NO | ✅ YES | Captures Stripe/SendGrid failure responses |
| Microservice Chain Timeout | ❌ NO | ✅ YES | Identifies exact failing service in chain |

